The Graph Has No Privileged Direction

The Intent Graph and Its Layers

The intent graph is a way of describing an organization — or any purposeful system — with enough precision that machines can reason over it. It has three layers.

The **intent layer** states what should be true. Not how, not when — just the declarative fact of what the system is oriented toward. Intents are durable. They change when purpose changes, not when implementation changes.

The **test layer** specifies what it means for an intent to be satisfied. Tests are the formal bridge between purpose and execution. They can be run, failed, and updated. They are the living contract between what you mean and what you've built.

The **expression layer** contains the actual artifacts — the running software, the deployed systems, the things you can point at. An expression either satisfies a test or it doesn't.

The standard picture has causality flowing downward: intents give rise to tests, tests constrain expressions. That picture is useful. It is also incomplete.

The Topology Is Richer Than the Hierarchy

Consider what happens when you start at the expression layer and look upward. A single artifact — a deployed service, a shared library, a data schema — might satisfy tests from two entirely separate intent stacks. Two groups, two sets of purposes, two test suites, all resolving to the same underlying thing.

The temptation is to treat this as a problem. Two stacks sharing an artifact implies coupling, and coupling implies the need for coordination or resolution. But this is a normative assumption smuggled in as a structural observation. There is no reason the two stacks need

to resolve. They can simply coexist. Each stack satisfies its own tests. The artifact is a node that happens to be reachable from multiple intent paths.

The only structural consequence is that the artifact becomes a higher-connectivity node in the graph. When it changes, both stacks receive that change as a ripple — independently, in their own terms. They do not need to coordinate. They do not need to converge. The graph registers the coupling that actually exists without requiring anyone to do anything about it.

The same topology operates in the other direction. A single intent does not require a unique test and implementation path. An intent states what should be true; it does not mandate how. Two implementation stacks can satisfy the same intent simultaneously, each with its own tests, its own artifacts, its own internal logic. Neither is primary. Neither is provisional. The intent has a family of satisfying implementations rather than a single canonical one.

This is not a degenerate case. It is ordinary. An organization that can authenticate users via OAuth and via SAML has two implementation stacks under one intent. Both are valid. The graph has no reason to privilege one over the other, and no mechanism for doing so — nor should it.

The upshot is that the graph has no privileged direction. You cannot read it strictly top-down as intents determining implementations, nor bottom-up as artifacts implying intents. The layers describe the character of nodes — their relationship to purpose, to specification, to execution — not positions in a causal chain. Edges describe satisfying relationships that can be one-to-many in either direction.

Fragments

Once you accept a non-hierarchical graph, the assumption that an organization has a single unified intent graph starts to look like the same mistake in a different form. It was sustainable only because the hierarchy seemed to require a single apex. Remove the apex, and there is no longer any reason for the enterprise to share one graph.

Each group — each team, each person, each organizational unit — can have its own graph fragment. A fragment is a self-contained subgraph: a set of intents, tests, and expressions that describe what that group is doing and what it cares about.

Fragments connect to each other wherever nodes happen to be shared. And the type of connection carries meaning.

When two fragments share an intent node, they are aligned on purpose. They may have entirely different test and implementation paths beneath that shared intent — different methods, different tools, different constraints. The connection says only that they care about the same thing. It does not say anything about how they work.

When two fragments share an artifact node, they have a dependency. They may have nothing in common at the intent level. One group produces the artifact; another consumes it. The shared node is the contract. Neither group needs to know the other's internal structure.

These are genuinely different kinds of inter-fragment relationship — alignment versus dependency — and the graph makes them visible without conflating them. Currently, both live informally in documentation, in conversations, in organizational memory. The graph makes them structural.

Across Organizational Boundaries

The graph does not know what an organizational boundary is. It knows only nodes and edges. There is no reason fragment connections must stop at the boundary of a legal entity or an org chart.

Shared artifacts across organizational boundaries are already universal. Open source libraries, shared APIs, industry standards — every organization depends on things it did

not build and does not control. What the graph adds is that this sharing becomes first-class structure rather than an informal dependency that lives outside your system description.

Shared intents across organizational boundaries are more interesting. If two organizations connect at an intent node, that connection says they are oriented toward the same thing. It requires nothing further. They do not need to share governance, methodology, tooling, or even awareness of each other's fragment internals. An industry consortium, a regulatory requirement, a common standard — these are all intent-layer connections between organizational fragments. The graph makes them legible in the same formalism used to describe purely internal structure.

Some nodes, followed to their logical conclusion, will turn out to be shared by many fragments across many organizations. These are effectively public infrastructure at the intent or artifact layer — the actual shared conceptual and technical substrate that an industry runs on. Currently this substrate is invisible, reconstructed from documentation and convention. The graph makes it a first-class structural fact.

The Client/Producer Case

The client/producer relationship is the fragment model at its clearest.

A producer exposes an artifact. A client's fragment connects to that artifact node. The client has intent and test layers above the shared artifact — the producer has intent and test layers behind it. Neither sees the other's internals. The shared node is the contract.

What the graph makes explicit that current practice leaves implicit: the client has tests against that artifact. Those tests are the client's actual requirements, stated formally and kept live. The producer has tests too, from their side. When the artifact changes, both test suites receive the ripple — independently, in their own terms.

This is a more honest description of what an API contract is than anything currently in use. An API specification describes the artifact's shape. It does not capture the client's intent for using it or the producer's intent for providing it. The graph carries both, connected at the artifact boundary.

When a producer wants to change an artifact, the graph tells them exactly which client fragments are connected and at what layer. Not through a registry, not through documentation maintained by hand, but structurally — the connectivity is the dependency manifest.

What the Shared Graph Produces

When fragments connect across organizational boundaries, both sides gain something they do not currently have.

Visibility. What is actually coupled becomes structurally apparent — not inferred from documentation or discovered when something breaks.

Legibility. The producer can read the client's actual intent, not just their usage patterns. Right now producers infer client needs from support tickets, API call volumes, and feature requests. If the client's intent layer is visible across the boundary, the producer sees why the artifact matters, not just how often it's called.

Communication. Shared nodes are shared vocabulary. When producer and client are both pointing at the same intent node, conversations stop being translation exercises.

Specification. Client tests against the artifact are the live requirements — not historical documents that degrade, not tickets that lose context. The specification is structural, not archival.

Value. A producer who can see which intents their artifact is serving across multiple client fragments knows where they are actually generating value — not where they think they

are. Investment follows actual structural importance rather than loudest voice or largest account.

All five of these currently exist as degraded, informal, lossy versions of what the graph makes precise. The graph does not create new relationships. It makes the existing ones legible for the first time.

The organizational boundary stops being a wall. It becomes a region of lower fragment connectivity — which is all it ever really was.